Tutorial:
Paso 1: Acceso al VMware

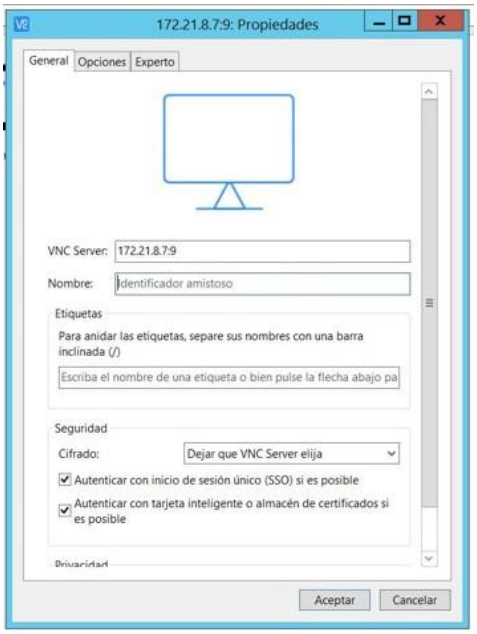
El flujo de herramientas se encuentra dentro del servidor de la Escuela de Ingeniería en Electrónica. Por lo tanto, se debe acceder al Vmware de la escuela desde cualquier browser. Aquí está el link:

https://virtual.ie.tec.ac.cr/

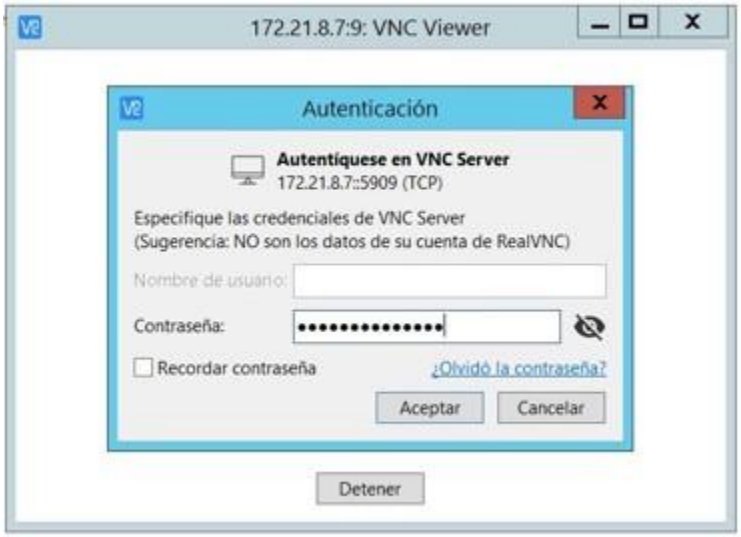
Deben introducir sus credenciales del servidor global de la Escuela en el dominio de estudiantes. Si no lo tienen, pueden solicitarlo con algún operador o con Ronald Soto.

Paso 2: Acceso al servidor por VNCViewer

Ingrese al servidor con sus credenciales usando VNCViewer (si tiene dudas con esta parte, consulte con algún profesor del DCILan, o revise, tutoriales previos). Introduzca la IP correspondiente:



Introduzca su contraseña de VNC para acceder al Sistema Operativo.



Paso 3: Copia de los scripts de compilación de programas

En la siguiente ruta se encuentran los scripts para compilar programas en ensamblador RISC-V32I y convertirlos a hexadecimal para poder ser corridos a nivel de simulación en el procesador SIWA, o para grabarlos en una memoria FLASH para correrlo en el procesador físico.

```
/mnt/vol_NFS_rh003/profesores/rmolina/RISCV_test_program_forge
[rmolina003@redhat003 RISCV_test_program_forge]$
```

Copie esa carpeta a su espacio de trabajo para poder editar sus contenidos. Una vez copiada la carpeta, encontrará dentro las siguientes carpetas de interés:

```
[rmolina003@redhat003 programs_verification]$ ls
Ensamblador  hexfiles  makefile  Objetos  scripts
```

De esas carpetas interesan:

- “Ensamblador”: donde se encuentran los programas en ensamblador.
- “Hexfiles”: donde se encuentran los archivos en hexadecimal después de ser compilados los programas.
- “Scripts”: donde se encuentran los scripts de compilación.

Las carpetas ensamblador, hexfiles y objetos tienen la siguiente estructura:

```
[rmolina003@redhat003 Ensamblador]$ ls
CPI_power  FPGA_tests  instrucciones_ interrupciones  Power_measure
```

Estas carpetas son solo una forma simbólica de ordenar los programas generados. Dentro de “Ensamblador”, cada una de estas carpetas contiene programas escritos en ensamblador. En cualquier lugar dentro de estas carpetas que se encuentre su programa, el mismo será compilado.

```
[rmolina003@redhat003 FPGA_tests]$ ls
arith_log_test.S              Estimulacion_RISCHV_V2.S  LED_Demo.S   makefile~        OLED_and_LEDs_Demo.S   OLED_test.S~
arith_log_test.S~             GPIO_simple_test.S        LED_Demo.S~  memory_tests.S   OLED_and_LEDs_Demo.S~
Estimulacion_RISCHV_V2_old.S  GPIO_simple_test.S~       makefile     memory_tests.S~  OLED_test.S
```

Paso 4: Creación y compilación.

Haga su programa en cualquiera de las carpetas ubicadas dentro de “Ensamblador”. Utilice cualquier editor de texto. En este tutorial se utiliza “gedit” o “sublime text” (comando:”subl”). Asegúrese que la extensión del archivo es *.S

```
.section .text
.globl _start
_start:
#Description:
#This program just loops NOP instructions to measure power consumption.
#-----------------------------------------------------------------------------
#-----------------------------------------------------------------------------
#NOPs loop
loop:
    nop
    nop
    nop
    nop
    nop
    nop
    nop
    nop
    nop
    nop
    nop
    nop
    nop
    nop
    nop
    j loop

```

```
[rmolina003@redhat003 FPGA_tests]$ ls
arith_log_test.S            Estimulacion_RISCHV_V2.S  LED_Demo.S   makefile~      nop_program.S      OLED_test.S
arith_log_test.S~           GPIO_simple_test.S        LED_Demo.S~  memory_tests.S  OLED_and_LEDs_Demo.S  OLED_test.S~
Estimulacion_RISCHV_V2_old.S  GPIO_simple_test.S~     makefile     memory_tests.S~ OLED_and_LEDs_Demo.S~
[rmolina003@redhat003 FPGA_tests]$
```

Una vez su programa esté listo. Diríjase a la carpeta “scripts”. Allí encontrará dos scripts de interés.

```
[rmolina003@redhat003 scripts]$ ls
ambientador.sh  compilacion.sh  libmpc.so.2  log.txt
```

- ambientador.sh: es un script que configura las variables de ambiente para ubicar el compilador.
- compilador.sh: es un script que realiza los pasos para compilar los programas.

Corra ambos scripts con el comando “source” (primero “ambientador.sh” y luego “compilador.sh”) para generar los archivos hexadecimales.

```
[rmolina003@redhat003 scripts]$ source ambientador.sh
[rmolina003@redhat003 scripts]$
```

```
riscv32-unknown-elf-gcc -nostdlib -nostartfiles -o ../../Objetos/CPI_power/bge.o bge.S
riscv32-unknown-elf-gcc -nostdlib -nostartfiles -o ../../Objetos/CPI_power/slt.o slt.S
riscv32-unknown-elf-gcc -nostdlib -nostartfiles -o ../../Objetos/FPGA_tests/OLED_test.o OLED_test.S
riscv32-unknown-elf-gcc -nostdlib -nostartfiles -o ../../Objetos/FPGA_tests/nop_program.o nop_program.S
riscv32-unknown-elf-gcc -nostdlib -nostartfiles -o ../../Objetos/FPGA_tests/Estimulacion_RISCHV_V2_old.o Estimulacion_RISCHV_V2_old.S
/mnt/vol_NFS_rh003/profesores/rmolina/tools_riscv/bin/../lib/gcc/riscv32-unknown-elf/8.2.0/../../../../riscv32-unknown-elf/bin/ld: warning:
cannot find entry symbol _start; defaulting to 0000000000010054
riscv32-unknown-elf-gcc -nostdlib -nostartfiles -o ../../Objetos/FPGA_tests/GPIO_simple_test.o GPIO_simple_test.S
riscv32-unknown-elf-gcc -nostdlib -nostartfiles -o ../../Objetos/FPGA_tests/arith_log_test.o arith_log_test.S
riscv32-unknown-elf-gcc -nostdlib -nostartfiles -o ../../Objetos/FPGA_tests/OLED_and_LEDs_Demo.o OLED_and_LEDs_Demo.S
riscv32-unknown-elf-gcc -nostdlib -nostartfiles -o ../../Objetos/FPGA_tests/Estimulacion_RISCHV_V2.o Estimulacion_RISCHV_V2.S
/mnt/vol_NFS_rh003/profesores/rmolina/tools_riscv/bin/../lib/gcc/riscv32-unknown-elf/8.2.0/../../../../riscv32-unknown-elf/bin/ld: warning:
cannot find entry symbol _start; defaulting to 0000000000010054
riscv32-unknown-elf-gcc -nostdlib -nostartfiles -o ../../Objetos/FPGA_tests/memory_tests.o memory_tests.S
riscv32-unknown-elf-gcc -nostdlib -nostartfiles -o ../../Objetos/FPGA_tests/LED_Demo.o LED_Demo.S
riscv32-unknown-elf-gcc -nostdlib -nostartfiles -o ../../Objetos/Power_measure/lw_med_power.o lw_med_power.S
riscv32-unknown-elf-gcc -nostdlib -nostartfiles -o ../../Objetos/Power_measure/sh_med_power.o sh_med_power.S
riscv32-unknown-elf-gcc -nostdlib -nostartfiles -o ../../Objetos/Power_measure/jal_med_power.o jal_med_power.S
riscv32-unknown-elf-gcc -nostdlib -nostartfiles -o ../../Objetos/Power_measure/addi_med_power.o addi_med_power.S
riscv32-unknown-elf-gcc -nostdlib -nostartfiles -o ../../Objetos/Power_measure/sw_med_power.o sw_med_power.S
riscv32-unknown-elf-gcc -nostdlib -nostartfiles -o ../../Objetos/Power_measure/beq_med_power.o beq_med_power.S
riscv32-unknown-elf-gcc -nostdlib -nostartfiles -o ../../Objetos/Power_measure/add_med_power.o add_med_power.S
riscv32-unknown-elf-gcc -nostdlib -nostartfiles -o ../../Objetos/Power_measure/lui_med_power.o lui_med_power.S
[rmolina003@redhat003 scripts]$
```

Si su programa tiene algún error de sintaxis, al correr compilador.sh aparecerán los mensajes de error.

Si el programa no tiene errores, el resultado debería aparecer en la carpeta correspondiente dentro de la carpeta “hexfiles”. El archivo tiene una extensión *.txt.

```
[rmolina003@redhat003 FPGA_tests]$ pwd
/mnt/vol_NFS_rh003/profesores/rmolina/RISCV_test_program_forge/programs_verification/hexfiles/FPGA_tests
[rmolina003@redhat003 FPGA_tests]$ ls
arith_log_test.o.txt    Estimulacion_RISCHV_V2_old.o.txt  GPIO_simple_test.o.txt~  mem_model.txt       OLED_and_LEDs_Demo.o.txt
binfiles                Estimulacion_RISCHV_V2.o.txt      how_to_txt_to_bin        memory.py           OLED_test.o.txt
convert_hex_to_bin.sh   Estimulacion_RISCHV_V2.o.txt.bin  LED_Demo.o.txt           memory_tests.o.txt  OLED_test.o.txt~
convert_hex_to_bin.sh~  GPIO_simple_test.o.txt            LED_Demo.o.txt~          nop_program.o.txt   x.txt
[rmolina003@redhat003 FPGA_tests]$
```

Paso 5: Preparación para simulación del programa

Para verificar que el programa hace lo que le corresponde, se puede simular a nivel de SystemVerilog el procesador con alguno de los programas compilados. Para ello, se van a ocupar las siguientes carpetas:

```
[rmolina003@redhat003 rmolina]$ pwd
/mnt/vol_NFS_rh003/profesores/rmolina
[rmolina003@redhat003 rmolina]$ ls
0.86                          Curso_Intel          GTKwave                    riscv_mcu                  Status.sv          tools
bash_snps_xt018-AMS_2017      Curso_Verif          GTKwave intallation_files  riscv_test                 sublime_text_3     tools_riscv
bash_snps_xt018-AMS_2017~     Digital_Designs      opendatabase.log           riscv_test_backup          Symbiotic_EDA      transcript
Capacitacion_Synopsys_2016    DVEfiles             Optimizacion               riscv_test_fpga_ver        Symbiyosys_tools   VCS_Examples
codigo_proyectos              entorno-uvm          programs_verification      RISCV_test_program_forge   Synopsys_Scripts
comandos                      evaristo_project     proyecto_Jill              riscv_test_regression      synopsys_tools.sh~
comandos~                     FPGA_8KB_Memory      riscv                      riscv_toolchain            test
cooper                        GNU_RISCV_Toolchain  riscv_2_0                  riscv_tools                tmp
[rmolina003@redhat003 rmolina]$
```

- Synopsys_Scripts: carpeta que contiene el script que configura las variables de ambiente de las herramientas de Synopsys.
- riscv_test_fpga_ver: carpeta que contiene la última versión del simulador.

Copie ambas carpetas a su espacio de trabajo, este proceso puede tardar algunos minutos.

Una vez copiadas ambas carpetas. Diríjase a la siguiente carpeta, que es donde se encuentra el simulador:

```
[rmolina003@redhat003 core_spi_uart]$ pwd
/mnt/vol_NFS_rh003/profesores/rmolina/riscv_test_fpga_ver/Verificacion_RISCV_TEC/test_env/core_spi_uart
[rmolina003@redhat003 core_spi_uart]$ ls
ac_check.err                      instruction_class.sv  results.txt       Status_post_synt.sv            topcore_tb.sv
cm.log                            inter.fsdb            run.sh            Status.sv                      uart_data.txt
comandos                          inter.vpd             sdfAnnotateInfo   Summary_Physical.txt           UART_Results
comandos~                         IS25WP032D.v          Selfcheck.sv      Summary_Physical.txt~          ucli.key
core_coverage_collection.sv       mem_model.txt         sim.txt           Summary_Pre-Synthesis.txt      urgReport
coverage_classes.sv               memory.py             simv              swave_compile_done             verdi_config_file
csrc                              Metrics_Reports       simv.daidir       temp.txt                       verdiLog
Dhrystone_Model.sv                novas.conf            simv.vdb          Test_Files                     what.txt
dhrystone_tb.sv                   novas_dump.log        stack.info.15143  tests.txt                      xxx.txt
DVEfiles                          novas.rc              stack.info.32035  tests.txt~
dve_report.031219.151219.tar.gz   opendatabase.log      stack.info.4908   top_core_spi_uart_phy.sv
dve_report.031219.151222.tar.gz   Reference_Model.sv    stack.info.5273   top_core_spi_uart_post_syn.sv
info.txt                          Results               stack.info.6441   top_core_spi_uart.sv
[rmolina003@redhat003 core_spi_uart]$ gedit comandos
```

De esos archivos, nos interesan los siguientes:

```
[rmolina003@redhat003 core_spi_uart]$ pwd
/mnt/vol_NFS_rh003/profesores/rmolina/riscv_test_fpga_ver/Verificacion_RISCV_TEC/test_env/core_spi_uart
[rmolina003@redhat003 core_spi_uart]$ ls
ac_check.err                      instruction_class.sv  results.txt       Status_post_synt.sv            topcore_tb.sv
cm.log                            inter.fsdb            run.sh            Status.sv                      uart_data.txt
comandos                          inter.vpd             sdfAnnotateInfo   Summary_Physical.txt           UART_Results
comandos~                         IS25WP032D.v          Selfcheck.sv      Summary_Physical.txt~          ucli.key
core_coverage_collection.sv       mem_model.txt         sim.txt           Summary_Pre-Synthesis.txt      urgReport
coverage_classes.sv               memory.py             simv              swave_compile_done             verdi_config_file
csrc                              Metrics_Reports       simv.daidir       temp.txt                       verdiLog
Dhrystone_Model.sv                novas.conf            simv.vdb          Test_Files                     what.txt
dhrystone_tb.sv                   novas_dump.log        stack.info.15143  tests.txt                      xxx.txt
DVEfiles                          novas.rc              stack.info.32035  tests.txt~
dve_report.031219.151219.tar.gz   opendatabase.log      stack.info.4908   top_core_spi_uart_phy.sv
dve_report.031219.151222.tar.gz   Reference_Model.sv    stack.info.5273   top_core_spi_uart_post_syn.sv
info.txt                          Results               stack.info.6441   top_core_spi_uart.sv
[rmolina003@redhat003 core_spi_uart]$ gedit comandos
```

- comandos: script que simula el procesador usando VCS y el simulador de RISC-V para validar el funcionamiento.

- mem_model.txt: archivo que se carga en el archivo de SystemVerilog de la memoria FLASH con el programa a correr en el procesador.
- memory.py: programa de Python que acomoda el archivo de la memoria para ser compatible con la memoria FLASH a nivel de SystemVerilog. Este programa genera automáticamente el archivo “mem_model.txt”.
- Test_Files: carpeta que contiene los programas en hexadecimal creados por el compilador (pasos anteriores).

Paso 6: Simulación del procesador.

Para simular el procesador, ocupamos en primera instancia copiar el programa que creamos a la carpeta donde se encuentran los programas del simulador.

```
[rmolina003@redhat003 Assembly_Code]$ pwd
/mnt/vol_NFS_rh003/profesores/rmolina/riscv_test_fpga_ver/Verificacion_RISCV_TEC/test_env/core_spi_uart/Test_Files/Assembly_Code
[rmolina003@redhat003 Assembly_Code]$ ls
all_tests  Aritmeticas  brincos  CSR  Estimulacion_RISCHV_V2.o.txt  power_meas  sencillo.o.txt~  Store_load
[rmolina003@redhat003 Assembly_Code]$
```

Podemos copiar nuestro programa ubicado en “hexfiles” a esta ubicación:

```
[rmolina003@redhat003 Assembly_Code]$ cp ../../../../../../RISCV_test_program_forge/programs_verification/hexfiles/FPGA_tests/nop_program.o.txt .
[rmolina003@redhat003 Assembly_Code]$ ls
all_tests  Aritmeticas  brincos  CSR  Estimulacion_RISCHV_V2.o.txt  nop_program.o.txt  power_meas  sencillo.o.txt~  Store_load
[rmolina003@redhat003 Assembly_Code]$
```

Posteriormente, debemos editar el script “comandos” para que pueda encontrar el programa y generar los archivos adecuados. Primero, edite la línea donde se corre el programa de Python para que encuentre el programa que queremos simular.

```
#Converting the Assembly Program into a code readable by the memory

#python memory.py Test_Files/Assembly_Code/Aritmeticas/add.o.txt

#python memory.py Test_Files/Assembly_Code/Store_load/Interrupt_UART_Bottom.o.txt

#python memory.py Test_Files/Assembly_Code/brincos/jump.o.txt

#python memory.py Test_Files/Assembly_Code/CSR/CSRRCI.o.txt

python3.9 memory.py Test_Files/Assembly_Code/nop_program.o.txt
```

Luego, edite la línea donde se simula el procesador, para que genere archivos de resultados con nombres representativos a los que estamos corriendo.

```
#Compiling the simulation

#vcs -sverilog -Mupdate -debug_all +vcs+flush+all +warn=all -timescale=1ns/10ps -full64 +incdir+../../TEC_RISCV/TOP +incdir+../../TEC_RISCV/SPI
+incdir+../../TEC_RISCV/UART top_core_spi_uart.sv

#vcs -ova_cov -cm line+cond+fsm+tgl+assert+branch+property_path -cm_report unencrypted_hierarchies+svpackages+noinitial -sverilog -lca -Mupdate -
debug_all +vcs+flush+all +warn=all -timescale=1ns/10ps -full64 +incdir+../../TEC_RISCV/TOP +incdir+../../TEC_RISCV/SPI +incdir+../../TEC_RISCV/UART -
CFLAGS -DVCS top_core_spi_uart.sv

#vcs -ova_cov -cm line+cond+fsm+tgl+assert+branch+property_path -cm_report unencrypted_hierarchies+svpackages+noinitial -sverilog -lca -Mupdate -
debug_all +vcs+flush+all +warn=all -timescale=1ns/10ps -full64 +incdir+../../TEC_RISCV/TOP +incdir+../../TEC_RISCV/SPI +incdir+../../TEC_RISCV/UART
top_core_spi_uart_post_syn.sv

#vcs -ova_cov -cm line+cond+fsm+tgl+assert+branch+property_path -cm_report unencrypted_hierarchies+svpackages+noinitial -sverilog -lca -Mupdate -
debug_all +vcs+flush+all +warn=all -timescale=1ns/10ps -full64 +incdir+../../TEC_RISCV/TOP +incdir+../../TEC_RISCV/SPI +incdir+../../TEC_RISCV/UART -
sdf typ:Test_Top.uut:../../TEC_RISCV/TOP/Tec_Riscv_pads_phy_no_pg.sdf +sdfverbose +neg_tchk top_core_spi_uart_phy.sv

#Verdi Test

vcs -kdb -ova_cov -cm line+cond+fsm+tgl+assert+branch+property_path -cm_report unencrypted_hierarchies+svpackages+noinitial -sverilog -lca -Mupdate -
debug_all +vcs+flush+all +warn=all -timescale=1ns/10ps -full64 -P ${VERDI_HOME}/share/PLI/VCS/linux64/novas.tab ${VERDI_HOME}/share/PLI/VCS/linux64/
pli.a +incdir+../../TEC_RISCV/TOP +incdir+../../TEC_RISCV/SPI +incdir+../../TEC_RISCV/UART -CFLAGS -DVCS top_core_spi_uart.sv


#Simulating the test
#./simv -cm line+cond+fsm+tgl+assert+branch+property_path -cm_name add +TESTNAME=add
#rm -rf csrc/ DVEfiles/ inter.vpd simv.daidir/ swave_compile_done ucli.key vcs.log verilog.dump #ac_check.err log

#Verdi Test
./simv -verdi -cm line+cond+fsm+tgl+assert+branch+property_path -cm_name nop +TESTNAME=nop_program
```

Guarde los resultados y cierre el programa. Ya podemos correr la simulación con el siguiente comando:

```
[rmolina003@redhat003 core_spi_uart]$ pwd
/mnt/vol_NFS_rh003/profesores/rmolina/riscv_test_fpga_ver/Verificacion_RISCV_TEC/test_env/core_spi_uart
[rmolina003@redhat003 core_spi_uart]$ ls
ac_check.err                    instruction_class.sv  results.txt       stack.info.6441                top_core_spi_uart.sv
cm.log                          inter.fsdb            run.sh            Status_post_synt.sv            topcore_tb.sv
comandos                        inter.vpd             sdfAnnotateInfo   Status.sv                      uart_data.txt
comandos~                       IS25WP032D.v          Selfcheck.sv      Summary_Physical.txt           UART_Results
core_coverage_collection.sv     mem_model.txt         sim_results.txt   Summary_Physical.txt~          ucli.key
coverage_classes.sv             memory.py             sim.txt           Summary_Pre-Synthesis.txt      urgReport
csrc                            Metrics_Reports       simv              swave_compile_done             verdi_config_file
Dhrystone_Model.sv              novas.conf            simv.daidir       temp.txt                       verdiLog
dhrystone_tb.sv                 novas_dump.log        simv.vdb          Test_Files                     what.txt
DVEfiles                        novas.rc              stack.info.15143  tests.txt                      xxx.txt
dve_report.031219.151219.tar.gz opendatabase.log      stack.info.32035  tests.txt~
dve_report.031219.151222.tar.gz Reference_Model.sv    stack.info.4908   top_core_spi_uart_phy.sv
info.txt                        Results               stack.info.5273   top_core_spi_uart_post_syn.sv
[rmolina003@redhat003 core_spi_uart]$ source comandos > sim_results.txt
[rmolina003@redhat003 core_spi_uart]$
```

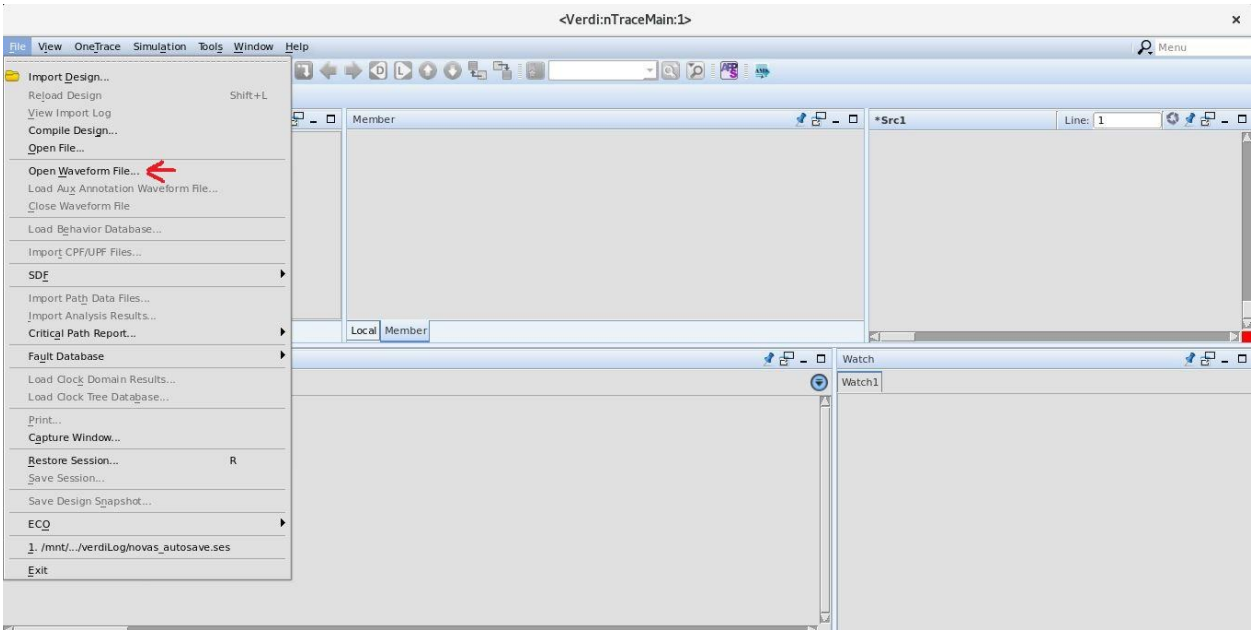
Ese comando genera el archivo de texto “sim_results.txt” con el análisis y resultados del simulador. Por ejemplo, la siguiente imagen presenta el resultado de nuestro programa. Aquí puede ver el comportamiento del Banco de Registros del procesador y del simulador, con ello se puede debuggear el código de nuestro programa.

```
x30, t5: 00000000
x31, t6: 00000000
------------------------------------

CSR,Mie/Mip/IO: 00010004
CSR,mepc: 00000000
CSR,Interrup1: 00000000
CSR,Interrup2: 00000000
CSR,GPIO: 00000000
CSR,mvtec: 00000000
CSR,timer_comp: 00000000
CSR,timer_valor: 00000000
CSR,full_shift: 00000000
CSR,IS_Val: 00000000
CSR,IS_Config: 00000000
CSR,IS_Trigger: 00000000
------------------------------------

----------------------------------------
----------------TEST PASS--------------
----------------------------------------
$finish called from file "top_core_spi_uart.sv", line 335.
$finish at simulation time           4461075000

------------------------------------------------------------------------------
VCS Coverage Metrics: during simulation line, cond, FSM, branch, tgl was monitored
------------------------------------------------------------------------------

 Coverage status: End of All Coverages ...

           V C S   S i m u l a t i o n   R e p o r t
Time: 4461075000 ps
CPU Time:      2.510 seconds;       Data structure size:   2.7Mb
Mon Nov 15 17:33:45 2021
```

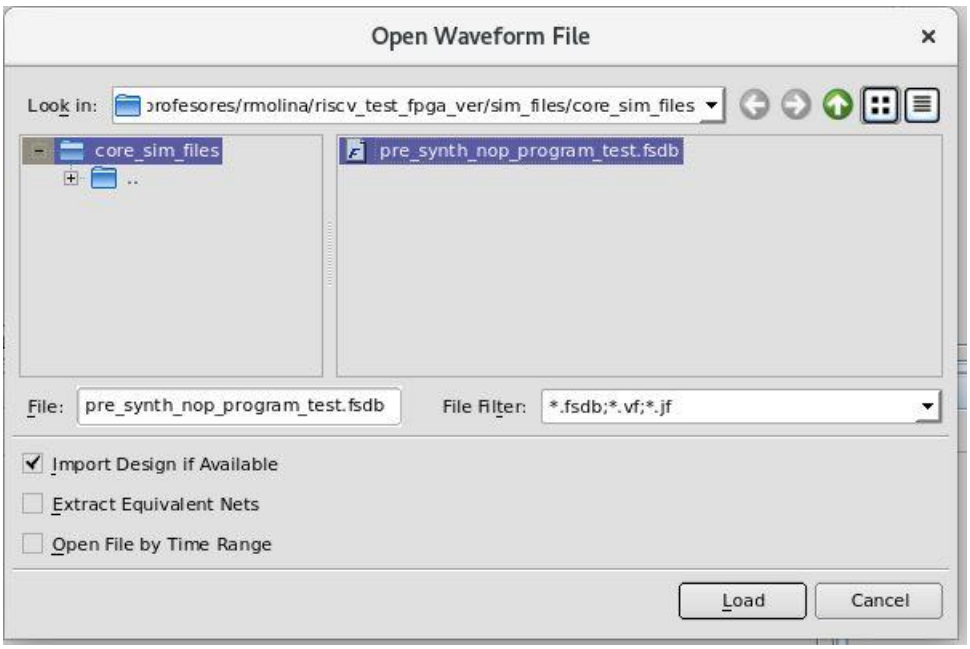
Si desea ver la simulación con las señales del procesador, los resultados se encuentran en:

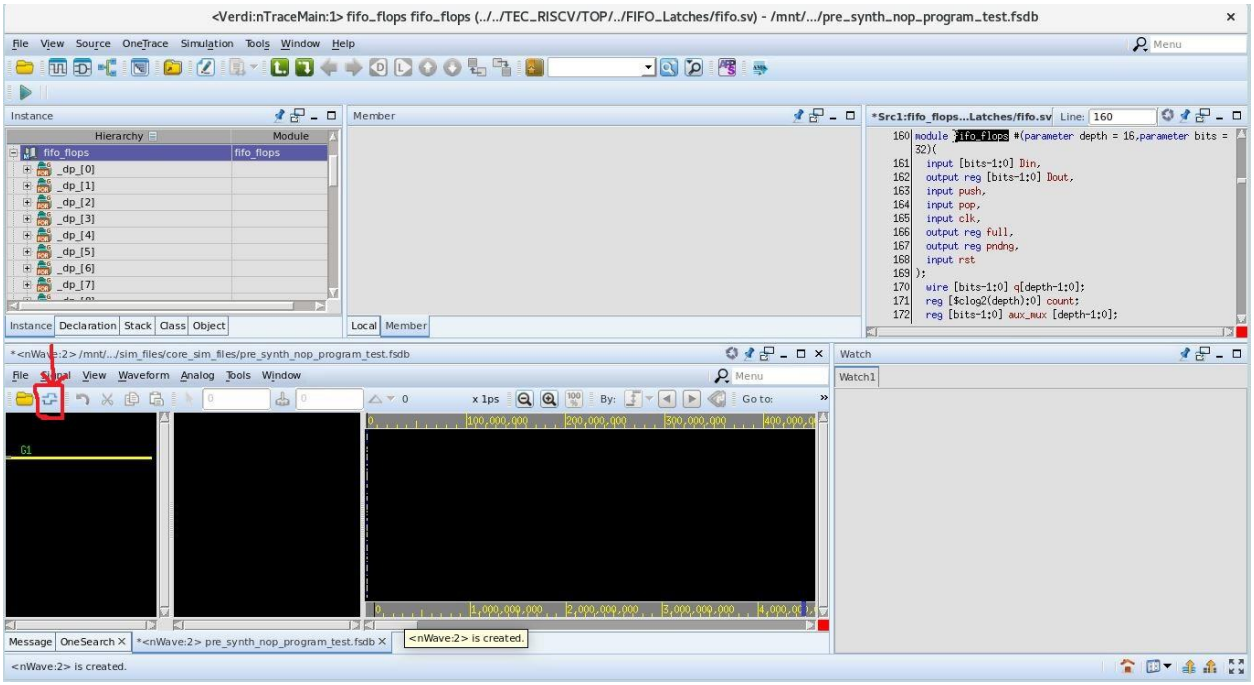
```
[rmolina003@redhat003 core_spi_uart]$ gedit sim_results.txt
[rmolina003@redhat003 core_spi_uart]$ ls ../../../sim_files/core_sim_files/
pre_synth_nop_program_test.fsdb  pre_synth_nop_program_test.vcd
[rmolina003@redhat003 core_spi_uart]$
```

Abra “Verdi” de Synopsys (comando: verdi) para visualizar los resultados:

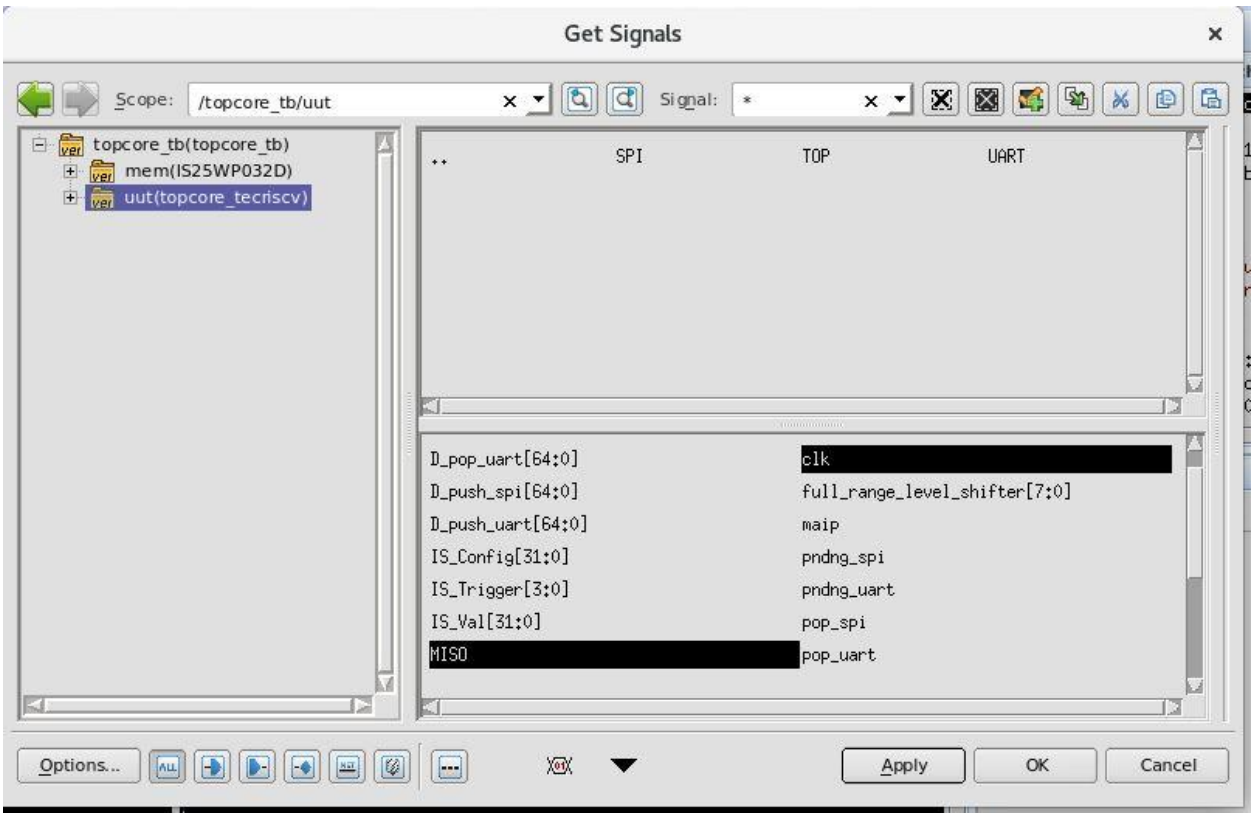


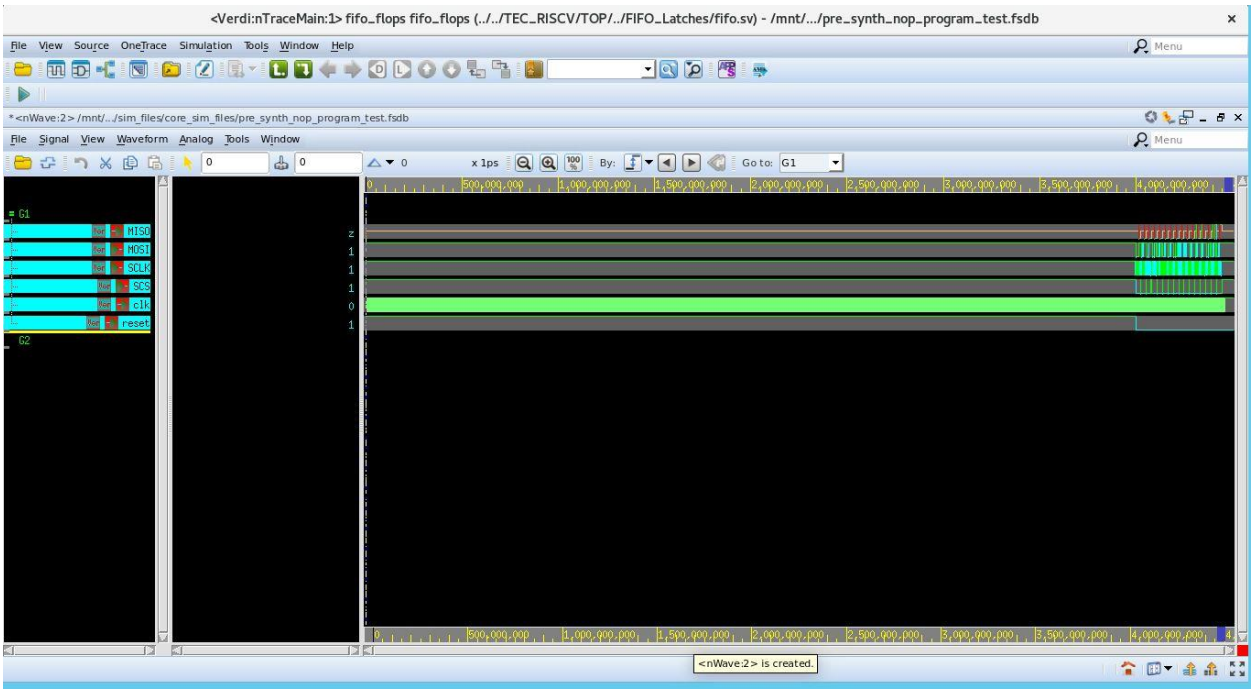
Seleccione el archivo *.fsdb.





Seleccione las señales que desea visualizar:




<Verdi:nTraceMain:1> fifo_flops fifo_flops (../../TEC_RISCV/TOP/../FIFO_Latches/fifo.sv) - /mnt/.../pre_synth_nop_program_test.fsdb
File View Source OneTrace Simulation Tools Window Help
*<nWave:2> /mnt/.../sim_files/core_sim_files/pre_synth_nop_program_test.fsdb
File Signal View Waveform Analog Tools Window
Go to: G1
MISO
MOSI
SCLK
SCS
clk
reset
<nWave:2> is created.

Paso 7: Generación del archivo *.bin para grabarlo en la memoria FLASH

Si el programa está listo, ocupamos el archivo *.bin para poder grabar la memoria FLASH con el programa que queremos correr en el procesador físico. Para ello, solo ocupamos correr los siguientes comandos:

```
[rmolina003@redhat003 FPGA_tests]$ pwd
/mnt/vol_NFS_rh003/profesores/rmolina/RISCV_test_program_forge/programs_verification/hexfiles/FPGA_tests
[rmolina003@redhat003 FPGA_tests]$ ls
arith_log_test.o.txt    Estimulacion_RISCHV_V2_old.o.txt  GPIO_simple_test.o.txt~  mem_model.txt       OLED_and_LEDs_Demo.o.txt
binfiles                Estimulacion_RISCHV_V2.o.txt      how_to_txt_to_bin        memory.py           OLED_test.o.txt
convert_hex_to_bin.sh   Estimulacion_RISCHV_V2.o.txt.bin  LED_Demo.o.txt           memory_tests.o.txt  OLED_test.o.txt~
convert_hex_to_bin.sh~  GPIO_simple_test.o.txt            LED_Demo.o.txt~          nop_program.o.txt   x.txt
[rmolina003@redhat003 FPGA_tests]$ python3.9 memory.py nop_program.o.txt
[rmolina003@redhat003 FPGA_tests]$ xxd -r -p mem_model.txt > nop_program.o.txt.bin
[rmolina003@redhat003 FPGA_tests]$ ls
arith_log_test.o.txt    Estimulacion_RISCHV_V2_old.o.txt  GPIO_simple_test.o.txt~  mem_model.txt       nop_program.o.txt.bin     x.txt
binfiles                Estimulacion_RISCHV_V2.o.txt      how_to_txt_to_bin        memory.py           OLED_and_LEDs_Demo.o.txt
convert_hex_to_bin.sh   Estimulacion_RISCHV_V2.o.txt.bin  LED_Demo.o.txt           memory_tests.o.txt  OLED_test.o.txt
convert_hex_to_bin.sh~  GPIO_simple_test.o.txt            LED_Demo.o.txt~          nop_program.o.txt   OLED_test.o.txt~
[rmolina003@redhat003 FPGA_tests]$
```